- skills/ — Reusable Analysis Functions for Agentic Specialists
  - Table of Contents
  - 1. What Is the Skills Library?
  - 2. Learner's Mental Model — A Toolbox
  - 3. Architecture Overview
    - Where Skills Fit in the System
    - Injection into execute_python
  - 4. The Standardized Return Contract
    - Why This Matters
  - 5. File Map
    - __init__.py — Package Entry Point
  - 6. Module 1: eda.py — Exploratory Data Analysis
    - Functions at a Glance
    - descriptive_stats(df, columns=None)
    - distribution_plots(df, columns=None, output_dir="output")
    - correlation_heatmap(df, columns=None, output_dir="output")
    - time_series_overview(df, columns=None, output_dir="output")
  - 7. Module 2: spc.py — Statistical Process Control
    - Functions at a Glance
    - control_limits(data, n_sigma=3.0)
    - xbar_chart(df, column, spec_limits=None, window='1h', n_sigma=3.0, output_dir="output")
    - process_capability(df, column, lsl, usl, target=None, output_dir="output")
    - control_limits_table(df, columns=None, n_sigma=3.0)
  - 8. Module 3: anomaly.py — Anomaly Detection & Regime Analysis
    - Functions at a Glance
    - isolation_forest_analysis(df, features=None, contamination=0.05, output_dir="output")
    - regime_detection(df, features=None, eps=0.8, min_samples=50, output_dir="output")
  - 9. Module 4: forecasting.py — Time Series Forecasting
    - Functions at a Glance
    - prophet_forecast(df, column, periods=1440, freq="1min", output_dir="output")
    - seasonal_decomposition(df, column, period=None, output_dir="output")
  - 10. Module 5: shift_kpi.py — Shift-Based KPI Analysis
    - Functions at a Glance
    - assign_shifts(df) → DataFrame
    - shift_kpis(df, columns=None, ore_col="Ore", downtime_threshold=10.0)
    - shift_comparison_chart(df, columns=None, output_dir="output")

    - downtime_analysis(df, ore_col="Ore", threshold=10.0, output_dir="output")
- 11. Module 6: optimization.py — Process Optimization
    - Functions at a Glance
    - pareto_frontier(df, x_col, y_col, x_minimize=False, y_minimize=True, output_dir="output")
    - sensitivity_analysis(df, target_col, feature_cols=None, output_dir="output")
    - optimal_windows(df, target_col, feature_cols=None, target_quantile=0.9, output_dir="output")
- 12. How Agents Use Skills
    - Typical Agent Flow
    - Integration with Domain Knowledge
    - Integration with Structured Output
- 13. Dependencies & Error Handling
    - Required Libraries
    - Matplotlib Backend
    - Minimum Data Guards
- 14. Quick Reference Table
- Summary

`skills/` — Reusable Analysis Functions for Agentic Specialists

> **Folder:** `python/agentic/skills/`
> **Role:** A library of pre-built, tested Python functions that AI agents call via `execute_python` instead of generating analysis code from scratch every time.

Table of Contents

1. What Is the Skills Library?
2. Learner's Mental Model — A Toolbox
3. Architecture Overview
4. The Standardized Return Contract
5. File Map
6. Module 1: `eda.py` — Exploratory Data Analysis
7. Module 2: `spc.py` — Statistical Process Control
8. Module 3: `anomaly.py` — Anomaly Detection & Regime Analysis
9. Module 4: `forecasting.py` — Time Series Forecasting

10. Module 5: `shift_kpi.py` — Shift-Based KPI Analysis
11. Module 6: `optimization.py` — Process Optimization
12. How Agents Use Skills
13. Dependencies & Error Handling
14. Quick Reference Table

---

1. What Is the Skills Library?

The `skills/` folder is a **package of reusable Python functions** designed for industrial process analysis. Instead of each AI agent writing analysis code from scratch (which is slow and error-prone), agents call these pre-built functions to:

- **Compute statistics** (mean, std, Cp/Cpk, correlations)
- **Generate charts** (control charts, heatmaps, time series, box plots)
- **Detect patterns** (anomalies, regimes, trends, seasonality)
- **Produce forecasts** (Prophet, seasonal decomposition)
- **Analyze operations** (shift KPIs, downtime events, optimal windows)

Every function returns a **standardized dict** — making it easy for agents to parse results and for downstream agents (like the reporter) to consume them.

---

2. Learner's Mental Model — A Toolbox

> **Think of a mechanic's toolbox.** A skilled mechanic doesn't forge their own wrench every time they need to tighten a bolt — they reach into a well-organized toolbox and grab the right tool. The skills library is that toolbox for AI agents.

```
┌──────────────────────────────────────────────────────────────────────┐
│                      THE TOOLBOX ANALOGY                             │
│                                                                      │
│  WITHOUT skills:                                                     │
│    Agent writes 50 lines of matplotlib code → might have bugs        │
│    Agent writes 30 lines of SPC code → might miscalculate Cpk        │
│    Agent writes anomaly detection → might forget to scale data       │
│    Each run: different code, different quality, unpredictable        │
│                                                                      │
│  WITH skills:                                                        │
│    Agent calls skills.spc.xbar_chart(df, 'PSI80')                    │
│    → Tested function, correct math, consistent output                │
│    → Returns {figures: [...], stats: {...}, summary: "..."}          │
│    → Agent uses the results instead of reinventing the wheel         │
│                                                                      │
```

```
│  Benefits:                                                          │
│    1. FASTER — one function call vs 50 lines of generated code      │
│    2. RELIABLE — tested functions, correct formulas every time      │
│    3. CONSISTENT — same output format, easy to chain together       │
│    4. READABLE — agents can reason about results, not debug code    │
└─────────────────────────────────────────────────────────────────────┘
```

---

3. Architecture Overview

Where Skills Fit in the System

```
┌──────────────────────────────────────────────────────────────────────┐
│                   HOW SKILLS ARE USED                                 │
│                                                                       │
│  User asks: "Analyze Mill 8 PSI80 quality"                            │
│       │                                                               │
│       ▼                                                               │
│  ┌──────────────────┐                                                 │
│  │  graph_v3.py     │  Planner selects: analyst, optimizer            │
│  │  (LangGraph)     │                                                 │
│  └──────────────────┘                                                 │
│           │                                                           │
│           ▼                                                           │
│  ┌──────────────────┐                                                 │
│  │  analyst agent   │  Generates Python code that CALLS skills:       │
│  │  (specialist)    │                                                 │
│  └──────────────────┘                                                 │
│           │                                                           │
│           ▼  execute_python tool                                      │
│  ┌──────────────────────────────────────────────────────────┐        │
│  │  Python sandbox (python_executor.py)                     │        │
│  │                                                          │        │
│  │  # Agent's generated code:                               │        │
│  │  result = skills.eda.descriptive_stats(df)               │        │
│  │  print(result['summary'])                                │        │
│  │                                                          │        │
│  │  result = skills.spc.xbar_chart(df, 'PSI80',             │        │
│  │      spec_limits=get_spec_limits('PSI80'),               │        │
│  │      output_dir=OUTPUT_DIR)                              │        │
│  │  print(result['summary'])                                │        │
│  │  print(f"Cpk = {result['stats']['Cpk']}")                │        │
│  └──────────────────────────────────────────────────────────┘        │
│           │                                                           │
│           ▼                                                           │
│  ┌──────────────────┐                                                 │
│  │  output/{id}/    │  Charts saved as .png files                     │
│  │  spc_xbar_PSI80  │                                                 │
│  │  .png            │                                                 │
│  └──────────────────┘                                                 │
└──────────────────────────────────────────────────────────────────────┘
```

Injection into execute_python

The `skills` module is injected into the Python executor's namespace in `python_executor.py`:

```
# In python_executor.py — namespace dict:
import skills as _skills_module

namespace = {
    # ... core libs (pd, np, plt, sns) ...
    "skills": _skills_module,       # ◄── THE SKILLS LIBRARY
    "PLANT_SPECS": PLANT_VARIABLES, # ◄── Domain knowledge
    "get_spec_limits": get_spec_limits,
    # ...
}
```

This means any agent code running inside `execute_python` can do:

```
import skills
result = skills.eda.descriptive_stats(df)
```

No installation needed — it's already in the namespace.

---

4. The Standardized Return Contract

Every function in the skills library returns the same dict structure:

```
{
    "figures": [str, ...],   # List of saved chart file paths (can be empty)
    "stats":   dict,          # Numeric results / structured data (can be empty)
    "summary": str,           # Human-readable text summary
}
```

Why This Matters

```
┌──────────────────────────────────────────────────────────────────────┐
│  THE STANDARD RETURN CONTRACT                                        │
│                                                                      │
│  ┌──────────┐      ┌──────────┐      ┌──────────┐                   │
│  │ figures  │      │  stats   │      │ summary  │                   │
│  │ (list)   │      │  (dict)  │      │  (str)   │                   │
```

```
│  └──────┬──────┘      └──────┬──────┘      └──────┬──────┘                    │
│         │                    │                    │                            │
│         ▼                    ▼                    ▼                            │
│  Reporter uses     Downstream       Agent reads                                │
│  paths to embed    agents parse     summary to                                 │
│  charts in the     numbers for      decide next                                │
│  final .md report  further analysis  steps                                     │
│                                                                                 │
│  Example:                                                                       │
│  {                                                                              │
│    "figures": ["output/abc123/spc_xbar_PSI80.png"],                             │
│    "stats": {"Cpk": 1.45, "UCL": 58.2, "LCL": 42.1},                            │
│    "summary": "SPC X̄ Chart for PSI80:\n  Cpk = 1.45 → CAPABLE"                 │
│  }                                                                              │
└─────────────────────────────────────────────────────────────────────────────────┘
```

Field	Type	When Empty	Used By
`figures`	`list[str]`	Pure computation, no chart	`reporter`, `code_reviewer`
`stats`	`dict`	Visualization-only functions	Downstream specialists
`summary`	`str`	Never — always has a summary	The calling agent itself

---

5. File Map

```
skills/
├── __init__.py        # Package entry — imports all 6 modules, defines __all__
├── eda.py             # Exploratory Data Analysis (4 functions)
├── spc.py             # Statistical Process Control (4 functions)
├── anomaly.py         # Anomaly Detection & Regime Analysis (3 functions)
├── forecasting.py     # Time Series Forecasting (2 functions)
├── shift_kpi.py       # Shift-Based KPI Analysis (4 functions)
└── optimization.py    # Process Optimization (3 functions)
                         ────────────────────────
                         Total: 20 functions
```

`__init__.py` — Package Entry Point

```python
"""
skills/ — Reusable analysis functions for agentic specialists.
"""

from skills import eda
from skills import spc
from skills import anomaly
from skills import forecasting
from skills import shift_kpi
```

```
from skills import optimization

__all__ = ["eda", "spc", "anomaly", "forecasting", "shift_kpi", "optimization"]
```

> **Key detail:** Each module is imported as a namespace — agents access functions as `skills.eda.descriptive_stats()`, not `from skills.eda import descriptive_stats`.

---

6. Module 1: `eda.py` — Exploratory Data Analysis

> **Purpose:** The first step in any analysis — understand the data before doing anything sophisticated. Compute basic statistics, plot distributions, find correlations, and visualize time series trends.

Functions at a Glance

Function	Generates Chart?	What It Computes
`descriptive_stats()`	No	mean, std, min, max, quartiles, missing % per col
`distribution_plots()`	Yes (.png)	Histograms + KDE for each numeric column
`correlation_heatmap()`	Yes (.png)	Correlation matrix + top 10 strongest pairs
`time_series_overview()`	Yes (.png)	Time series line plots + 1h rolling mean overlay

`descriptive_stats(df, columns=None)`

Computes summary statistics for every numeric column. This is the **first function most agents call** — it gives them an overview of the data.

```
# Agent code inside execute_python:
result = skills.eda.descriptive_stats(df)
print(result['summary'])
```

```
# Output:
# Descriptive Statistics (14 variables, 10080 rows):
#   Ore: mean=185.3, std=22.1, range=[140.0, 220.0], missing=0.5%
#   PSI80: mean=48.7, std=5.2, range=[35.1, 62.3], missing=1.2%
#   ...
```

How it works:

```
┌──────────────────────────────────────────────────────────┐
│  descriptive_stats(df, columns=None)                      │
│                                                           │
│  1. If columns=None → auto-select all numeric columns     │
│  2. For each column:                                      │
│     - Drop NaN values                                     │
│     - Compute: count, mean, std, min, max, q25, q50, q75│
│     - Compute: missing_pct = (NaN count / total) * 100   │
│  3. Build summary text lines                              │
│  4. Return {figures: [], stats: {...}, summary: "..."}    │
│                                                           │
│  Note: No chart — pure computation. Fast.                 │
└──────────────────────────────────────────────────────────┘
```

Stats output structure:

```
stats = {
    "Ore": {
        "count": 10030,
        "mean": 185.3421,
        "std": 22.0891,
        "min": 140.0,
        "max": 220.0,
        "q25": 168.5,
        "q50": 185.0,   # median
        "q75": 202.1,
        "missing_pct": 0.5,
    },
    "PSI80": { ... },
    # ...one entry per column
}
```

`distribution_plots(df, columns=None, output_dir="output")`

Generates a grid of histograms — one per numeric column — with a **red dashed mean line** on each.

```
result = skills.eda.distribution_plots(df, output_dir=OUTPUT_DIR)
# Saves: output/{id}/distribution_plots.png
```

Chart layout logic:

```
┌──────────────────────────────────────────────────────┐
│  Grid: max 3 columns wide, rows as needed            │
│                                                      │
│  14 variables → 3 cols × 5 rows = 15 subplots       │
│  (last subplot hidden if empty)                      │
│                                                      │
│  Each subplot:                                       │
│     ┌──────────────────────────┐                     │
│     │  ▇▇▇▇▇▇▇                 │ ← histogram         │
│     │  ▇▇▇▇▇▇▇▇▇▇              │   (50 bins)         │
│     │  ▇▇▇▇▇▇▇    │            │                     │
│     │  ▇▇▇▇▇▇     │←── red     │ ← mean line         │
│     │  ▇▇▇▇       │    dashed  │                     │
│     │  ▇▇                      │                     │
│     └──────────────────────────┘                     │
│    Title: column name (bold)                         │
│    Legend: "Mean: 185.34"                            │
└──────────────────────────────────────────────────────┘
```

`correlation_heatmap(df, columns=None, output_dir="output")`

Produces a **lower-triangle correlation matrix** heatmap with annotated values, plus extracts the top 10 strongest correlations as structured data.

```
result = skills.eda.correlation_heatmap(df, output_dir=OUTPUT_DIR)
print(result['stats']['top_correlations'][:3])
# [{'var1': 'WaterMill', 'var2': 'DensityHC', 'correlation': -0.8234},
#  {'var1': 'Ore', 'var2': 'MotorAmp', 'correlation': 0.7891},
#  {'var1': 'PressureHC', 'var2': 'PulpHC', 'correlation': 0.6543}]
```

Key implementation details:

```
# Uses upper-triangle mask for clean visualization
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, vmin=-1, vmax=1, ...)
```

- Colormap: `RdBu_r` (red = negative, blue = positive, white = zero)

- Annotated values: 2 decimal places
- Top correlations sorted by |r| descending

`time_series_overview(df, columns=None, output_dir="output")`

Plots up to 8 variables stacked vertically with a **1-hour rolling mean** overlay in red.

```
result = skills.eda.time_series_overview(df, columns=['Ore', 'PSI80', 'WaterMill'],
                                          output_dir=OUTPUT_DIR)
```

```
┌──────────────────────────────────────────────────┐
│  Time Series Overview                            │
│                                                  │
│  Ore      ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒  (blue, alpha=0.5)  │
│           ────────────────────── (red rolling mean)  │
│  ────────────────────────────────────────────────│
│  PSI80    ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒                   │
│           ────────────────────                   │
│  ────────────────────────────────────────────────│
│  Water    ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒                   │
│           ────────────────────                   │
│                                                  │
│  Shared X axis (time), each variable gets its own │
│  Y axis. Max 8 variables to keep chart readable. │
└──────────────────────────────────────────────────┘
```

---

7. Module 2: `spc.py` — Statistical Process Control

> **Purpose:** Quality control analysis. Calculate control limits, generate X-bar charts, compute process capability indices (Cp/Cpk), and determine if a process is "in control."

> **What is SPC?** SPC uses statistics to monitor a process and detect when it drifts out of its normal operating range. The key question: *"Is this process stable and capable of meeting specifications?"*

Functions at a Glance

Function	Generates Chart?	What It Computes
`control_limits()`	No	CL, UCL, LCL for a single series
`xbar_chart()`	Yes (.png)	Full X-bar control chart with spec limits + Cpk
`process_capability()`	Yes (.png)	Histogram + normal fit + Cp/Cpk/PPM analysis
`control_limits_table()`	No	Summary table of control limits for all variables

`control_limits(data, n_sigma=3.0)`

The simplest function — computes 3 numbers:

```
limits = skills.spc.control_limits(df['PSI80'])
# Returns: {"CL": 48.72, "UCL": 64.31, "LCL": 33.13, "std": 5.19}
```

```
┌──────────────────────────────────────┐
│  Control Limits Concept              │
│                                      │
│  UCL = mean + 3σ ── Upper danger zone │
│  ─ ─ ─ ─ ─ ─ ─ ─                    │
│                                      │
│  CL  = mean ──────── Center line     │
│                                      │
│  ─ ─ ─ ─ ─ ─ ─ ─                    │
│  LCL = mean - 3σ ── Lower danger zone │
│                                      │
│  Points outside UCL/LCL = "out of    │
│  control" = process is unstable      │
└──────────────────────────────────────┘
```

`xbar_chart(df, column, spec_limits=None, window='1h', n_sigma=3.0, output_dir="output")`

The **centerpiece** of SPC analysis. Generates a full X-bar control chart with:

- Raw data (light blue, thin)

- X-bar rolling mean (navy, medium)
- CL / UCL / LCL lines (green / red dashed)
- Optional spec limits: LSL / USL (orange dotted)
- Out-of-control points highlighted (red dots)
- Cpk value in the title

```
# Agent code — using domain knowledge for spec limits:
lsl, usl = get_spec_limits('PSI80')  # → (40, 60)
result = skills.spc.xbar_chart(df, 'PSI80',
                                spec_limits=(lsl, usl),
                                window='1h',
                                output_dir=OUTPUT_DIR)
print(result['summary'])
# SPC X̄ Chart for PSI80:
#   CL=48.72, UCL=64.31, LCL=33.13, σ=5.19
#   Out-of-control: 127 points (1.26%)
#   Cpk = 1.450 → CAPABLE
```

Stats returned:

```
{
    "CL": 48.72,    "UCL": 64.31,   "LCL": 33.13,   "sigma": 5.19,
    "out_of_control_count": 127,     "out_of_control_pct": 1.26,
    "total_points": 10080,
    "Cpk": 1.45,    "LSL": 40,       "USL": 60,       # only if spec_limits given
}
```

Cpk interpretation (built-in):

```
Cpk ≥ 1.33  →  "CAPABLE"       (process fits well within specs)
Cpk ≥ 1.00  →  "MARGINAL"      (process barely meets specs)
Cpk <  1.00  →  "NOT CAPABLE"  (process exceeds spec limits)
```

process_capability(df, column, lsl, usl, target=None, output_dir="output")

A deeper capability study. Generates a **histogram with normal curve overlay** and computes Cp, Cpk, and PPM (parts per million) out of spec.

```
result = skills.spc.process_capability(df, 'PSI80', lsl=40, usl=60,
output_dir=OUTPUT_DIR)
print(result['stats'])
```

```
# {'Cp': 1.52, 'Cpk': 1.45, 'mean': 48.72, 'std': 2.19,
#  'ppm_total': 3.4, 'actual_out_of_spec_count': 0, ...}
```

Chart anatomy:

```
┌──────────────────────────────────────────────────────────┐
│  Process Capability — PSI80 (Cpk = 1.450, Cp = 1.520)    │
│                                                          │
│           LSL              Target              USL       │
│            │     ┌──────┐     │                  │      │
│            │     │ ▓▓▓▓ │     │                  │      │
│            │   ▓▓▓▓▓▓▓▓▓▓     │                  │      │
│            │  ▓▓▓▓▓▓▓▓▓▓▓▓    │                  │      │
│            │ ▓▓▓▓─Normal─fit─│──────────────── │        │
│            │  ▓▓▓▓▓▓▓▓▓▓▓▓    │                  │      │
│            │   ▓▓▓▓▓▓▓▓▓▓     │                  │      │
│            │     │ ▓▓▓▓ │     │                  │      │
│            │     └──────┘     │                  │      │
│            ▼                  ▼                  ▼      │
│     orange dashed       green line       orange dashed    │
│                                                          │
│  Red line = mean, Red curve = normal fit                 │
└──────────────────────────────────────────────────────────┘
```

`control_limits_table(df, columns=None, n_sigma=3.0)`

Batch computes control limits for all variables — useful for a quick overview:

```
result = skills.spc.control_limits_table(df)
# stats = {"Ore": {"CL": 185.3, "UCL": 251.4, "LCL": 119.2, "std": 22.0}, ...}
```

---

8. Module 3: `anomaly.py` — Anomaly Detection & Regime Analysis

> **Purpose:** Find unusual data points and identify distinct operating states. Uses machine learning (Isolation Forest, DBSCAN) rather than simple threshold rules.

> **What is anomaly detection?** Instead of manually setting "alert if value > X", these algorithms learn what "normal" looks like from the data itself, then flag points that

> deviate from that pattern — even in multivariate space (multiple variables simultaneously).

Functions at a Glance

Function	Generates Chart?	What It Does
`isolation_forest_analysis()`	Yes (.png)	ML anomaly detection + feature importance + timeline
`anomaly_timeline()`	Yes (.png)	Visualize pre-computed anomaly labels on time series
`regime_detection()`	Yes (.png)	DBSCAN clustering → operating regime identification

`isolation_forest_analysis(df, features=None, contamination=0.05, output_dir="output")`

The main anomaly detection function. Uses **Isolation Forest** — an algorithm that isolates anomalies by randomly partitioning data. Anomalies are easier to isolate, so they get separated in fewer partitions.

```
result = skills.anomaly.isolation_forest_analysis(df, contamination=0.05,
                                                   output_dir=OUTPUT_DIR)
print(result['summary'])
# Isolation Forest Anomaly Detection:
#   Anomalies: 504 / 10080 (5.0%)
#   Features: Ore, WaterMill, WaterZumpf, PSI80, DensityHC, PressureHC, ...
#   Top contributors: PSI80, DensityHC, MotorAmp, Ore, WaterMill
```

Processing pipeline:

```
┌──────────────────────────────────────────────────────────────────┐
│  isolation_forest_analysis() — Step by Step                      │
│                                                                  │
│  1. Select features (all numeric if None)                        │
│  2. Drop NaN rows → df_clean                                     │
│  3. StandardScaler().fit_transform(df_clean) → X_scaled          │
│     (centers data to mean=0, std=1 — essential for IF)           │
│                                                                  │
│  4. IsolationForest(contamination=0.05).fit_predict(X_scaled)    │
```

```
│     → labels:  1 = normal, -1 = anomaly                          │
│     → scores: lower = more anomalous                              │
│                                                                   │
│  5. Feature importance via mean absolute score difference:        │
│     For each feature:                                             │
│       importance = |mean(anomaly_vals) - mean(normal_vals)|      │
│     → Shows which variables contribute most to anomalies          │
│                                                                   │
│  6. Generate multi-panel chart:                                   │
│     ┌── top 3 features (time series + red anomaly dots) ──┐      │
│     │   Ore        ──●──────●──────   (● = anomaly)        │      │
│     │   PSI80      ──────●──●──────                        │      │
│     │   DensityHC  ──────●──────●──                        │      │
│     ├── anomaly score (purple line, red threshold) ────────┤      │
│     │   Score      ─────▼──────▼───   (dips = anomalies)   │      │
│     └──────────────────────────────────────────────────────┘      │
│                                                                   │
│  7. Return {figures, stats, summary}                              │
└───────────────────────────────────────────────────────────────────┘
```

regime_detection(df, features=None, eps=0.8, min_samples=50, output_dir="output")

Uses **DBSCAN clustering** to find distinct operating regimes — groups of data points that represent different states of the process (e.g., "high throughput" vs "low quality mode" vs "startup").

```
result = skills.anomaly.regime_detection(df, output_dir=OUTPUT_DIR)
print(result['summary'])
# Operating Regime Detection (DBSCAN eps=0.8, min_samples=50):
#   Regimes found: 3, Noise points: 245
#   Regime 0: 5200 pts (52.3%) — Ore=190.5, PSI80=49.2, DensityHC=1.65
#   Regime 1: 3100 pts (31.1%) — Ore=155.3, PSI80=44.1, DensityHC=1.58
#   Regime 2: 1535 pts (15.4%) — Ore=210.8, PSI80=52.7, DensityHC=1.72
```

Chart: Scatter plot of first two features colored by regime (tab10 colormap).

DBSCAN parameters:

- `eps` (0.8) — radius of a neighborhood. Smaller = tighter clusters
- `min_samples` (50) — minimum points to form a cluster. Larger = fewer, bigger clusters
- Points that don't belong to any cluster → labeled as "noise" (-1)

---

9. Module 4: `forecasting.py` — Time Series Forecasting

> **Purpose:** Predict future values and understand the underlying structure (trend, seasonality, residuals) of time series data.

Functions at a Glance

Function	Generates Chart?	What It Does
`prophet_forecast()`	Yes (.png)	Facebook Prophet forecast + trend + changepoints
`seasonal_decomposition()`	Yes (.png)	Additive decomposition → trend/seasonal/residual

`prophet_forecast(df, column, periods=1440, freq="1min", output_dir="output")`

Uses **Facebook Prophet** to forecast future values with confidence intervals and automatic changepoint detection.

```
result = skills.forecasting.prophet_forecast(df, 'PSI80', periods=480, freq='1min',
                                              output_dir=OUTPUT_DIR)
print(result['summary'])
# Prophet Forecast for PSI80:
#   Trend: increasing (+2.3%)
#   Changepoints detected: 7
#   Forecast (480 periods): mean=49.8, CI=[44.2, 55.4]
```

Two-panel chart:

```
┌──────────────────────────────────────────────────────┐
│  Panel 1: Prophet Forecast — PSI80 (increasing, +2.3%)│
│                                                       │
│  ▒▒▒▒▒▒▒▒▒▒▒▒▒▒ actual data (blue)                     │
│                ──────── forecast (red)                │
│                ▒▒▒▒▒▒▒▒ 95% CI (light red fill)      │
│  | | |        changepoints (gray dashed verticals)   │
│                                                       │
│  Panel 2: Trend Component                             │
```

```
│  ──────────────────────── smooth trend (navy)          │
└──────────────────────────────────────────────────────────┘
```

Key parameters:

- `periods` — how many future data points to predict (default: 1440 = 24 hours at 1min frequency)
- `freq` — time frequency of the data
- Prophet auto-detects: daily seasonality, weekly seasonality, trend changepoints

`seasonal_decomposition(df, column, period=None, output_dir="output")`

Breaks a time series into its building blocks: **trend + seasonal pattern + residual noise**.

```
result = skills.forecasting.seasonal_decomposition(df, 'Ore', output_dir=OUTPUT_DIR)
print(result['stats'])
# {'period': 1440, 'seasonal_strength': 0.342, 'trend_strength': 0.178}
```

Four-panel chart:

```
┌──────────────────────────────────────────────────────────┐
│  Observed  ▒▒▒
```

> **Purpose:** Analyze performance across the plant's 3-shift schedule. Compute KPIs per shift, compare shifts visually, and identify downtime events.

> **Domain context:** The Ellatzite plant operates on a **3-shift schedule**:
>
> - Shift 1: 06:00–14:00
> - Shift 2: 14:00–22:00
> - Shift 3: 22:00–06:00 (overnight)

Functions at a Glance

Function	Generates Chart?	What It Does
`assign_shifts()`	No	Adds `shift` (1/2/3) and `shift_date` columns
`shift_kpis()`	No	Computes mean/std/uptime/throughput per shift
`shift_comparison_chart()`	Yes (.png)	Box plots comparing shifts for each variable
`downtime_analysis()`	Yes (.png)	Identifies & visualizes downtime events

`assign_shifts(df) → DataFrame`

A helper that adds shift labels based on the hour of the DatetimeIndex:

```
df_with_shifts = skills.shift_kpi.assign_shifts(df)
# New columns: 'shift' (1, 2, or 3), 'shift_date' (date object)
```

```
# Shift assignment logic:
hours = df.index.hour
df["shift"] = np.where(
    (hours >= 6) & (hours < 14), 1,        # Shift 1: 06-14
    np.where((hours >= 14) & (hours < 22), 2, 3)  # Shift 2: 14-22, Shift 3: 22-06
)
```

`shift_kpis(df, columns=None, ore_col="Ore", downtime_threshold=10.0)`

Computes per-shift statistics including uptime/downtime based on ore flow:

```
result = skills.shift_kpi.shift_kpis(df)
print(result['stats']['shift_1'])
# {'label': 'Shift 1 (06-14)', 'data_points': 3360,
#  'Ore': {'mean': 192.5, 'std': 18.3, 'min': 140.0, 'max': 220.0},
#  'uptime_pct': 97.2, 'downtime_pct': 2.8, 'throughput_mean': 193.4}
```

Uptime/downtime logic:

```
┌──────────────────────────────────────────────────────────┐
│  Uptime detection: is the mill running?                  │
│                                                          │
│  If Ore ≥ threshold (10 t/h) → mill is RUNNING          │
│  If Ore <  threshold (10 t/h) → mill is DOWN             │
│                                                          │
│  uptime_pct = running_points / total_points × 100        │
│  throughput_mean = mean Ore DURING running periods only  │
└──────────────────────────────────────────────────────────┘
```

shift_comparison_chart(df, columns=None, output_dir="output")

Generates a grid of **box plots** comparing the 3 shifts side by side for each variable:

```
┌──────────────────────────────────────────────────────────┐
│  Shift Comparison (box plots)                            │
│                                                          │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐               │
│  │   Ore    │  │  PSI80   │  │ WaterMill│               │
│  │ ┌┐ ┌┐ ┌┐ │  │ ┌┐ ┌┐ ┌┐ │  │ ┌┐ ┌┐ ┌┐ │               │
│  │ │1│ │2│ │3│ │  │ │1│ │2│ │3│ │  │ │1│ │2│ │3│ │               │
│  │ └┘ └┘ └┘ │  │ └┘ └┘ └┘ │  │ └┘ └┘ └┘ │               │
│  └──────────┘  └──────────┘  └──────────┘               │
│                                                          │
│  Colors: Shift 1 = Blue, Shift 2 = Orange, Shift 3 = Green  │
└──────────────────────────────────────────────────────────┘
```

downtime_analysis(df, ore_col="Ore", threshold=10.0, output_dir="output")

Finds **contiguous downtime events** (periods where Ore < threshold) and computes their durations.

```
result = skills.shift_kpi.downtime_analysis(df, threshold=10, output_dir=OUTPUT_DIR)
print(result['summary'])
# Downtime Analysis (threshold = 10 t/h):
#   Events: 5, Total downtime: 12.3 hours (7.3%)
#   #1: 2026-03-27 02:15, duration 4.5h
#   #2: 2026-03-29 14:30, duration 3.2h
```

Two-panel chart:

```
┌──────────────────────────────────────────────────────┐
│  Panel 1: Ore timeline with downtime regions          │
│  ▒▒▒
```

> **Purpose:** Find the best operating conditions. Identify trade-offs between competing objectives, measure which inputs matter most, and discover the operating windows where the target performs best.

Functions at a Glance

Function	Generates Chart?	What It Does
`pareto_frontier()`	Yes (.png)	Find & plot Pareto-optimal trade-off curve
`sensitivity_analysis()`	Yes (.png)	Tornado chart of feature impact on target
`optimal_windows()`	Yes (.png)	Compare feature distributions in top vs all data

`pareto_frontier(df, x_col, y_col, x_minimize=False, y_minimize=True, output_dir="output")`

Finds the **Pareto frontier** — the set of points where you can't improve one objective without worsening the other.

```
result = skills.optimization.pareto_frontier(df, 'Ore', 'PSI80',
                                              x_minimize=False,  # more Ore = better
                                              y_minimize=True,   # lower PSI80 = better
                                              output_dir=OUTPUT_DIR)
print(result['summary'])
# Pareto Frontier: Ore vs PSI80
#   Pareto-optimal points: 23 / 10080 total
#   Best trade-offs (first 5):
#     Ore=215.30, PSI80=41.20
#     Ore=212.50, PSI80=42.10
#     ...
```

Chart:

```
┌──────────────────────────────────────────────┐
│  Pareto Frontier: Ore vs PSI80               │
│                                              │
│  PSI80 ▲                                     │
│        │    ·  ·  ·                          │
```

```
│        │  ·  ●──●  ·  ·    ← red Pareto line│
│        │ ·  ●    ·  ·  ·                   │
│        │·  ●  ·   ·  ·  ·  · = all points   │
│        │  ●  ·  ·  ·  ·                    │
│        │  · ·  ·  ·  ·  ·  ● = Pareto-optimal│
│        └──────────────────────► Ore         │
└──────────────────────────────────────────────┘
```

sensitivity_analysis(df, target_col, feature_cols=None, output_dir="output")

Creates a **tornado chart** showing how strongly each feature correlates with the target — revealing which input variables have the biggest impact.

```
result = skills.optimization.sensitivity_analysis(df, 'PSI80', output_dir=OUTPUT_DIR)
# Sensitivity Analysis for PSI80:
#   WaterMill: r=-0.823 (negative)
#   Ore: r=0.645 (positive)
#   DensityHC: r=-0.521 (negative)
```

Chart:

```
┌──────────────────────────────────────────────────────┐
│  Sensitivity Analysis (Tornado) -- PSI80             │
│                                                      │
│  WaterMill  ═══════════════  -0.82   (red = negative)│
│  Ore        ═══════════      +0.65   (green = positive)│
│  DensityHC  ═══════          -0.52                   │
│  PressureHC ══════           +0.41                   │
│  MotorAmp   ═══              +0.28                   │
│             ────┼──────────────►                     │
│                 0       correlation                  │
└──────────────────────────────────────────────────────┘
```

optimal_windows(df, target_col, feature_cols=None, target_quantile=0.9, output_dir="output")

Answers: *"When the target is at its best (top 10%), what do the input variables look like?"*

Compares the distribution of each feature in the **top quantile** of target performance vs the full dataset.

```
result = skills.optimization.optimal_windows(df, 'PSI80', target_quantile=0.9,
                                              output_dir=OUTPUT_DIR)
print(result['stats']['optimal_ranges']['Ore'])
# {'low': 175.2, 'high': 210.5, 'full_mean': 185.3, 'top_mean': 193.7}
```

Chart: Grid of overlaid histograms (blue = all data, green = top X%):

```
┌──────────────────────────────────────────────────────────┐
│  Optimal Operating Windows (top 90% of PSI80)            │
│                                                          │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐   │
│   │ Ore          │  │ WaterMill    │  │ DensityHC    │   │
│   │ ▓▓▓▓ blue    │  │ ▓▓▓▓ blue    │  │ ▓▓▓▓ blue    │   │
│   │ ▓▓ green     │  │ ▓▓ green     │  │ ▓▓ green     │   │
│   │ │ optimal ||│  │ │ optimal ||│  │ │ optimal ||│   │
│   └──────────────┘  └──────────────┘  └──────────────┘   │
│                                                          │
│  Red dashed lines = optimal range (10th-90th             │
│  percentile of top-performing data)                      │
└──────────────────────────────────────────────────────────┘
```

---

12. How Agents Use Skills

Typical Agent Flow

```
┌──────────────────────────────────────────────────────────────────┐
│  Agent: "analyst" specialist                                     │
│                                                                  │
│  Step 1: Understand the data                                     │
│    result = skills.eda.descriptive_stats(df)                     │
│    print(result['summary'])                                      │
│                                                                  │
│  Step 2: Visualize distributions                                 │
│    result = skills.eda.distribution_plots(df, output_dir=OUTPUT_DIR)│
│    print(result['summary'])                                      │
│                                                                  │
│  Step 3: Find correlations                                       │
│    result = skills.eda.correlation_heatmap(df, output_dir=OUTPUT_DIR)│
│    top_corr = result['stats']['top_correlations'][:3]            │
│    print(f"Strongest: {top_corr}")                               │
│                                                                  │
│  Step 4: SPC analysis (using domain knowledge)                   │
│    lsl, usl = get_spec_limits('PSI80')                           │
│    result = skills.spc.xbar_chart(df, 'PSI80',                   │
│                                   spec_limits=(lsl, usl),        │
│                                   output_dir=OUTPUT_DIR)         │
│    cpk = result['stats']['Cpk']                                  │
│    print(f"Process capability: Cpk={cpk}")                       │
```

```
│                                                                  │
│  Each step:                                                      │
│    - Calls a tested skill function                               │
│    - Gets structured results                                     │
│    - Prints summary for context                                  │
│    - Charts auto-saved to output/{analysis_id}/                  │
└──────────────────────────────────────────────────────────────────┘
```

Integration with Domain Knowledge

Skills work best **combined with domain knowledge** from `tools/domain_knowledge.py`:

```python
# PLANT_SPECS and get_spec_limits are pre-loaded in the execute_python namespace

# Get the official plant spec limits for a variable
lsl, usl = get_spec_limits('PSI80')  # → (40, 60)

# Use them in SPC analysis
result = skills.spc.process_capability(df, 'PSI80', lsl=lsl, usl=usl,
                                       output_dir=OUTPUT_DIR)
# Now Cp/Cpk are calculated against the REAL plant specifications
# instead of arbitrary guesses
```

Integration with Structured Output

Skills results can be emitted as structured output for downstream agents:

```python
import json
result = skills.spc.xbar_chart(df, 'PSI80', spec_limits=(40, 60), output_dir=OUTPUT_DIR)
# Emit structured data for the reporter:
print(f"STRUCTURED_OUTPUT:{json.dumps(result['stats'])}")
```

---

13. Dependencies & Error Handling

Required Libraries

All modules use these **core** libraries (always available):

Library	Import	Used For
`numpy`	`import numpy as np`	Numeric operations

Library	Import	Used For
`pandas`	`import pandas as pd`	DataFrame manipulation
`matplotlib`	`import matplotlib.pyplot as plt`	Chart generation
`seaborn`	`import seaborn as sns`	Heatmaps (eda.py only)

Some modules use **optional** libraries with graceful fallback:

Library	Module	Used For	Fallback
`sklearn`	anomaly.py	IsolationForest, DBSCAN, StandardScaler	Returns early with message
`scipy.stats`	spc.py	Normal distribution for PPM	Only in `process_capability`
`prophet`	forecasting.py	Time series forecasting	Returns "Prophet not installed"
`statsmodels`	forecasting.py	Seasonal decomposition	Returns "statsmodels not installed"

Matplotlib Backend

Every module sets the Agg (non-interactive) backend at import time:

```
import matplotlib
matplotlib.use("Agg")  # Non-interactive — required for server-side chart generation
```

This is **critical** because the code runs in a headless server process — there is no display to show interactive charts.

Minimum Data Guards

Every function checks for sufficient data before proceeding:

```
# Examples of minimum data checks:
if len(data) < 10:    # xbar_chart
if len(data) < 30:    # process_capability  (need ≥30 for meaningful Cp/Cpk)
if len(data) < 50:    # isolation_forest, optimal_windows
if len(data) < 100:   # prophet_forecast, seasonal_decomposition
if len(df_clean) < min_samples * 2:  # regime_detection
```

When data is insufficient, functions return an empty result with an explanatory summary:

```
return {"figures": [], "stats": {}, "summary": f"Insufficient data ({len(data)}
points)."}
```

---

14. Quick Reference Table

Module	Function	Chart	Key Output	Min Data
eda	descriptive_stats()	—	mean/std/min/max/quartiles	1
eda	distribution_plots()	.png	Histogram grid	1
eda	correlation_heatmap()	.png	Top 10 correlations	2
eda	time_series_overview()	.png	Rolling mean overlay	1
spc	control_limits()	—	CL, UCL, LCL, std	2
spc	xbar_chart()	.png	Control chart + Cpk	10
spc	process_capability()	.png	Cp, Cpk, PPM, histogram	30
spc	control_limits_table()	—	Multi-variable limits summary	2
anomaly	isolation_forest_analysis()	.png	Anomaly labels + importance	50
anomaly	anomaly_timeline()	.png	Pre-computed anomaly viz	1
anomaly	regime_detection()	.png	DBSCAN clusters + stats	100
forecasting	prophet_forecast()	.png	Forecast + CI + trend	100
forecasting	seasonal_decomposition()	.png	Trend/seasonal/residual	100
shift_kpi	assign_shifts()	—	DataFrame + shift column	1
shift_kpi	shift_kpis()	—	Per-shift mean/std/uptime	1
shift_kpi	shift_comparison_chart()	.png	Box plots by shift	1
shift_kpi	downtime_analysis()	.png	Events + durations + timeline	1
optimization	pareto_frontier()	.png	Pareto-optimal trade-offs	10

Module	Function	Chart	Key Output	Min Data
`optimization`	`sensitivity_analysis()`	.png	Tornado chart (correlations)	10
`optimization`	`optimal_windows()`	.png	Top-quantile operating ranges	50

Summary

The `skills/` library provides **20 tested functions across 6 modules** that turn agents from "code generators" into "analysis tool users." Key design principles:

1. **Standardized contract** — every function returns `{figures, stats, summary}`
2. **Self-contained** — each function handles its own plotting, computation, and error checking
3. **Domain-aware** — designed for industrial process data (mill sensors, shifts, SPC)
4. **Graceful degradation** — missing libraries or insufficient data → clear message, no crash
5. **Agent-friendly** — summaries are human-readable, stats are machine-parseable